

COS214 Practical 5

Courtesy of:

1. P (Patrick) Simuyemba u25632354
2. SE (Shelby) Bodenstein u25038967

Task 1: Analyse the Scenario and Design the System

a. Incidence-Response responsibilities

CampusGuard supports an incident from registration until it is closed. An incident is registered against a campus area in the **New** state. The operator can then:

- issue a public alert;
- dispatch a response unit to that area;
- change access to the area (lock, unlock, or restrict).

Dispatching a unit coordinates the other response services (security, medical, facilities, communications) so those units do not call each other. Incident status and area access change through defined states; invalid changes are refused.

A high-level workflow can run several of these steps in sequence (activate or resolve). Alerts leave the system through the existing legacy alert interface. When the incident is **Resolved**, the area can be unlocked and an all-clear issued.

b. UML Diagram

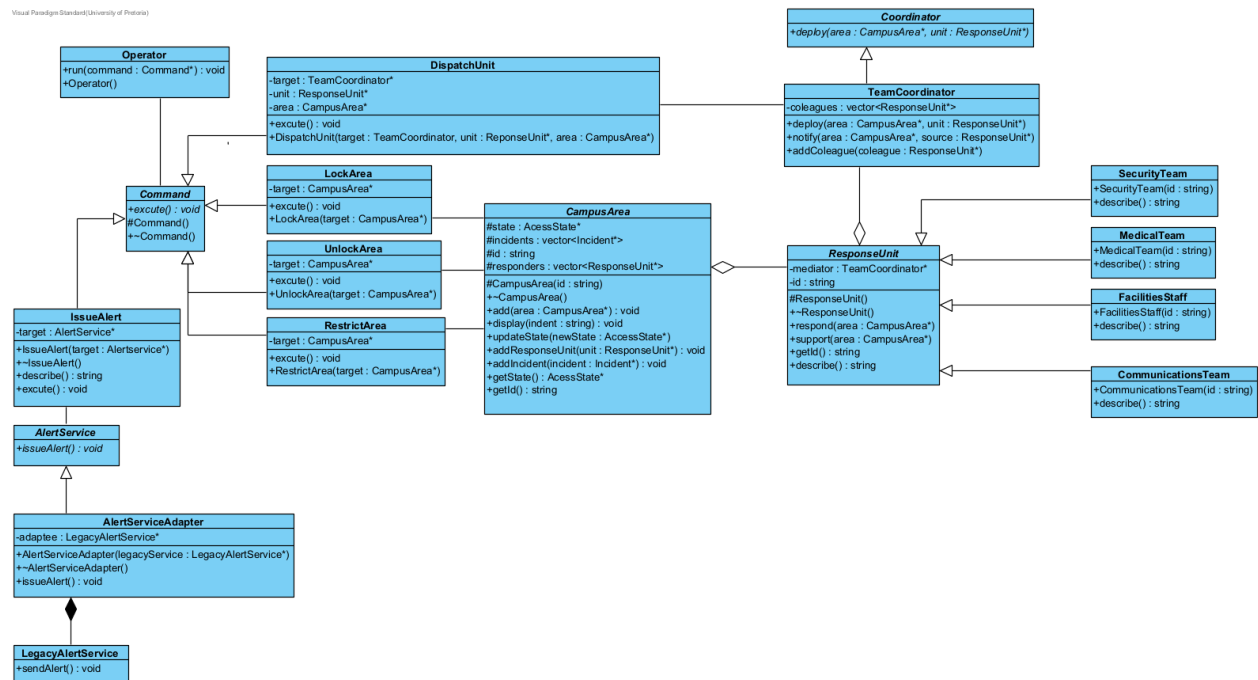


Figure 1: Command, Mediator and Adapter

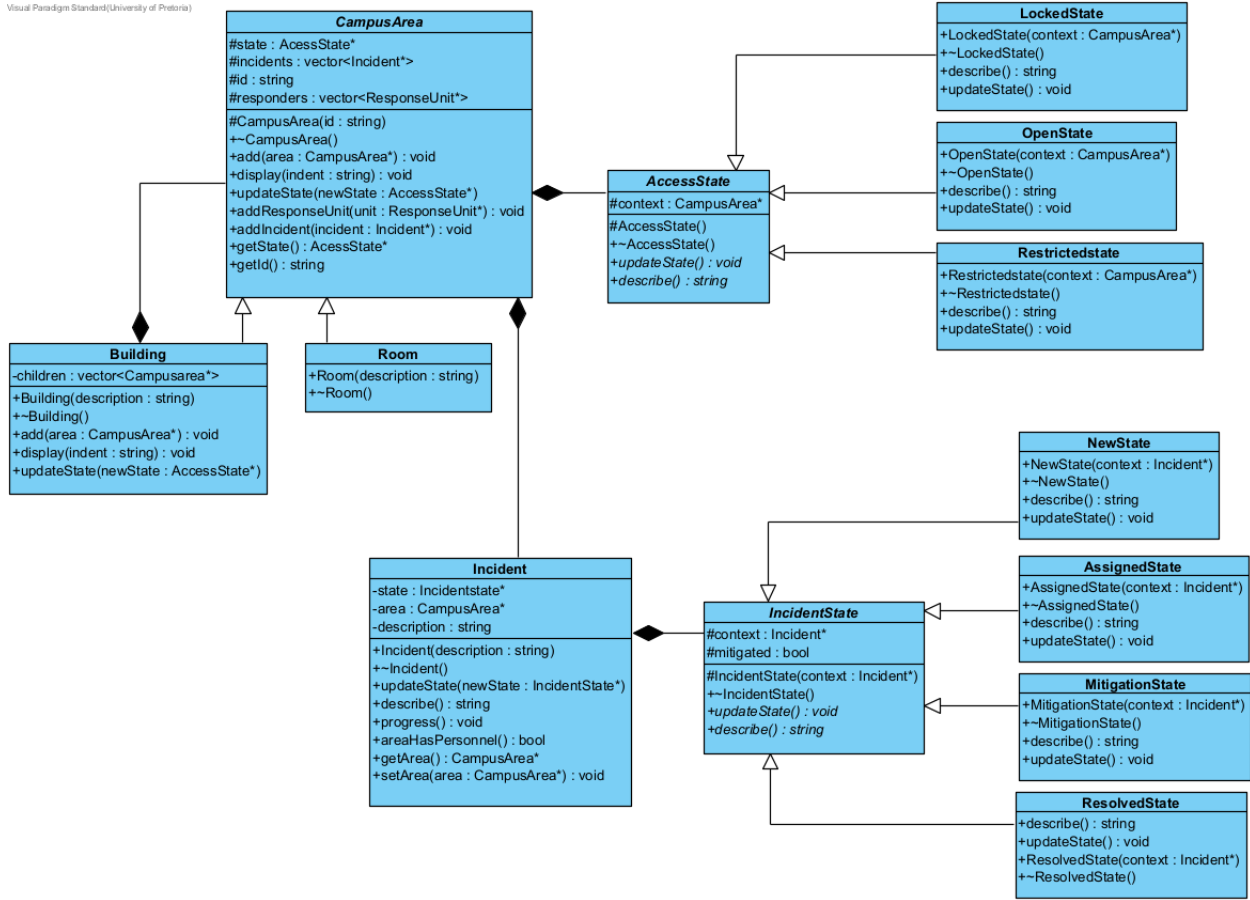


Figure 2: Areas, access state and the incident states

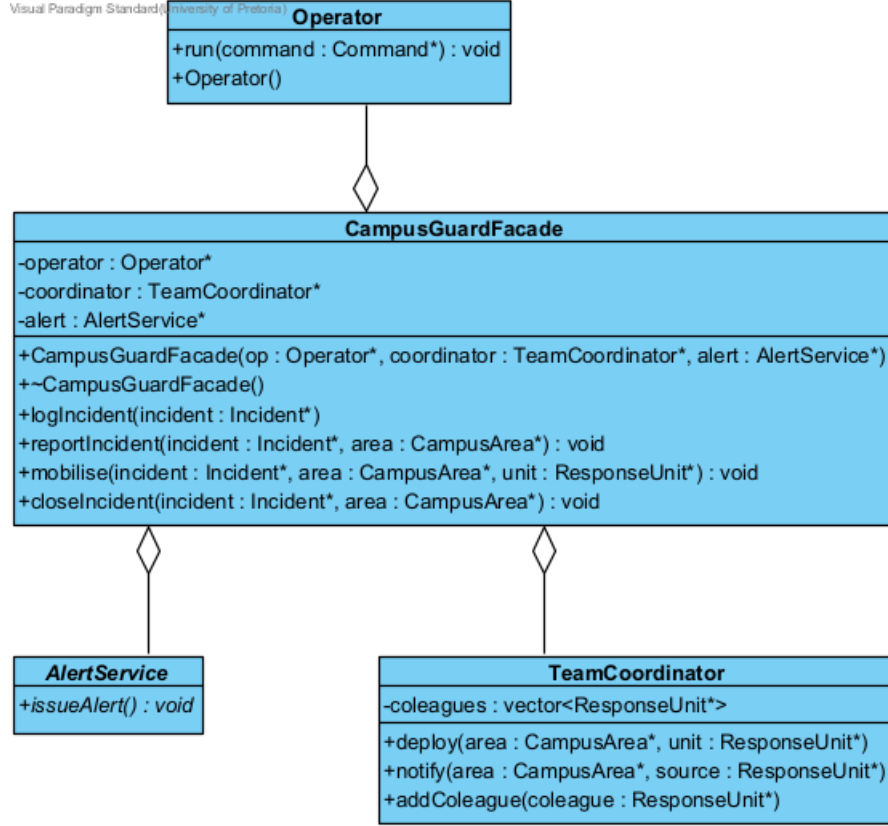


Figure 3: CampusGuardFacade with Operator, AlertService and TeamCoordinator

c. GoF Participants in named patterns

I. Command

- Invoker: Operator
- Receiver: CampusArea, TeamCoordinator, AlertService
- Command: Command
- ConcreteCommand: LockArea, UnlockArea, RestrictArea, DispathUnit, IssueAlert

II. Mediator

- Mediator: Coordinator
- ConcreteMediator: TeamCoordinator
- Colleague: ResponseUnit
- ConcreteColleague: CommunicationsTeam, SecurityTeam, FacilitiesTeam, MedicalTeam

III. Facade

- Facade: CampusQuardFacade

IV. Adapter

- Adapter: AlertServiceAdapter
- Adaptee: LegacyAlertService
- Target: AlertService

V. State

Justification: CampusGuard has two lifecycles whose legal next step depends on the current status: an incident (New, Assigned, Mitigation, Resolved) and an area's access (Open, Restricted, Locked). A simple `enum` plus `if/switch` in `Incident` or `CampusArea` would put every transition in one place, grow with every new status, and break the brief's rule against large centralised conditionals. State moves each transition into its own class. The current object accepts or refuses the change (for example New stays New until personnel are on the area; Resolved refuses another progress; Unlock refuses if the building is already open).

In the workflow, `progress()` and access commands (`RestrictArea`, `LockArea`, `UnlockArea`) call the current state. Scenario 1 assigns the fire only after Security is dispatched, then mitigates and resolves it, while the Engineering Building moves Restricted and later Open. The same machines run under the facade in Scenario 2.

- State: `IncidentState`, `AccessState`
- ConcreteState: `NewState`, `AssignedState`, `MitigationState`, `ResolvedState`, `OpenState`, `RestrictedState`, `LockedState`
- Context: `Incident`, `CampusArea`

VI. Composite

Justification: A campus is not a flat list of rooms. Operators lock or restrict a whole building and expect every hall and lab inside it to follow. Storing rooms in a vector and looping in `main` or in each command would duplicate that walk and force clients to know the tree. Composite lets a `Building` and a `Room` share the `CampusArea` interface. `RestrictArea` (and lock/unlock) is issued once on the building; the composite forwards the access state to its children.

In the workflow, Engineering (Lab, Lecture Room 2-1, Server Room) and Science (Chemistry Lab, Physics Lab) are the two composites. Display prints the tree so we can see one command change every child. Incidents are recorded on the area; dispatch still targets that area, whether it is a room or a building.

- Component: `CampusArea`
- Composite: `Building`
- Leaf: `Room`

Task 2: Implement the Integrated Model

a. Completed

Task 3: Build Two End-to-End Scenarios

a. Scenario 1: Fire in the Engineering Building

The Engineering Building is a composite: Engineering Lab, Lecture Room 2-1 and the Server Room. A fire incident is recorded on that building in `NewState`.

The operator first tries to progress the incident. It moves New to Assigned. A second progress is refused because security is not on site yet (invalid mitigation).

`DispatchUnit` then sends Campus Security to the building. `Operator` executes the command; `TeamCoordinator` deploys Security and notifies Medical, Facilities and Communications so those colleagues do not call each other.

`RestrictArea` is run on the whole Engineering Building. Composite forwards the access change to every room (`RestrictedState`). `IssueAlert` is run next; `AlertServiceAdapter` translates it for `LegacyAlertService`.

With a unit on site, the incident can progress `NewState` to Assigned to Mitigation, then Mitigation to Resolved. `UnlockArea` reopens the building and its rooms.

Patterns in this flow: Command, Mediator, Adapter, State and Composite (five of the six).

b. Scenario 2: Chemical leak in the Science Building

The Science Building is a second composite (Chemistry Lab and Physics Lab) with its own incident, "Chemical leak in Chemistry Lab". The client does not assemble those steps itself. It uses `CampusGuardFacade`.

reportIncident logs the leak, attaches it to Science, progresses New to Assigned, restricts the building, and issues an alert through the adapter. A mitigate attempt before Medical arrives is refused.

mobilise then dispatches the Medical Team through the coordinator (Command and Mediator again) so the leak can move to Mitigation. A second **LockArea** on Science is refused while the building is already closed. **closeIncident** resolves the leak and unlocks Science.

Patterns in this flow: Facade plus Command, Mediator, Adapter, State and Composite. Facade is the entry point; the same subsystems as Scenario 1 remain usable on their own.

Task 4: UML Diagram Portfolio

Class diagram: Command, Mediator and Adapter

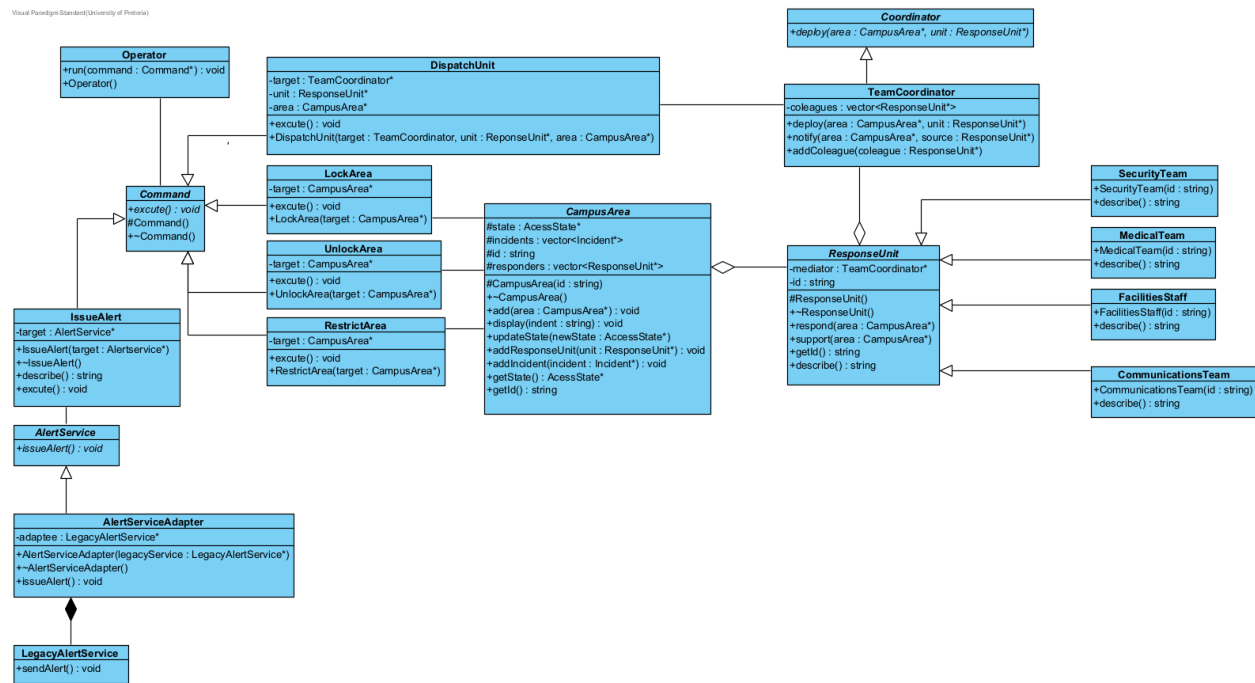


Figure 4: Command, Mediator and Adapter

Class diagram: areas, access state and incident states

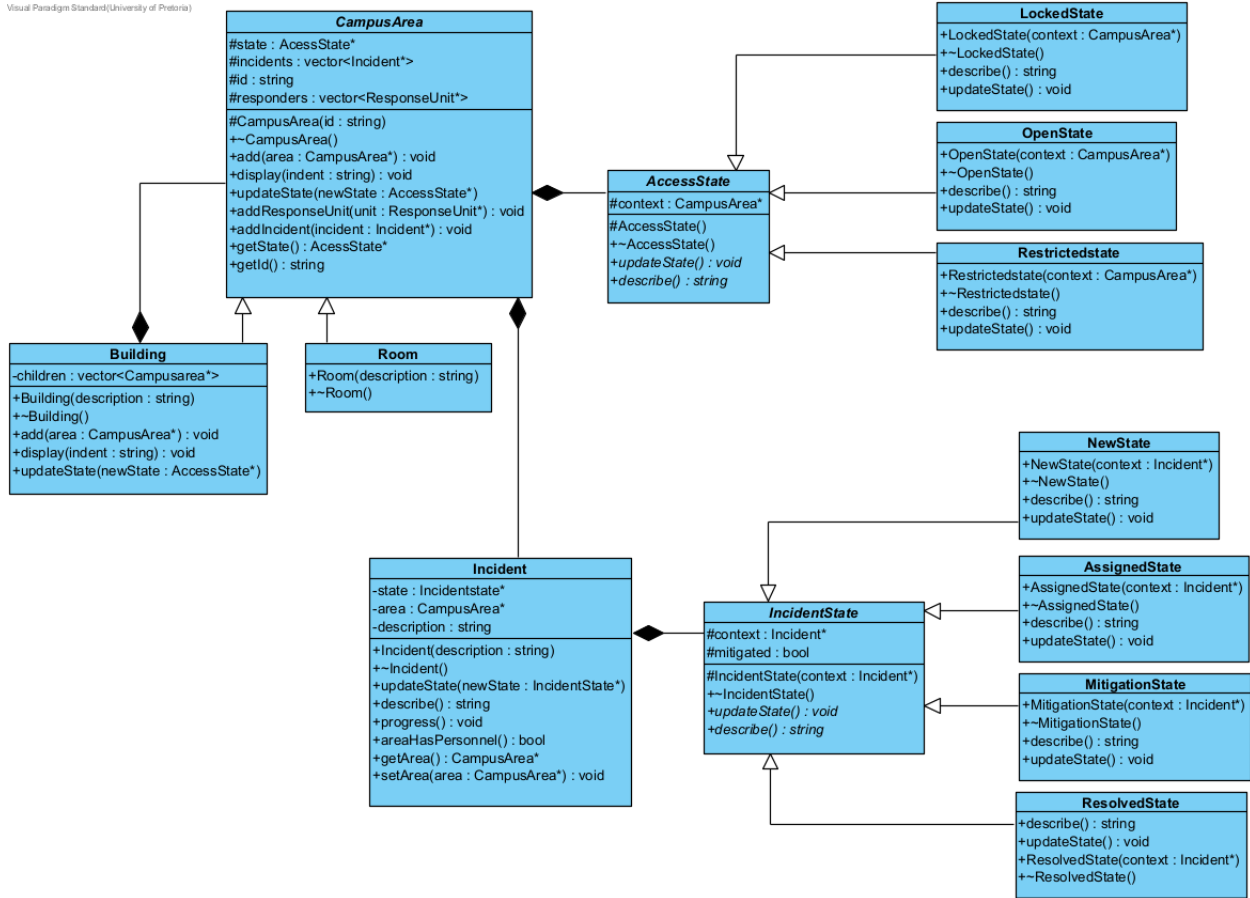


Figure 5: Areas, access state and the incident states

Class diagram: Facade, operator, alert service and team coordinator

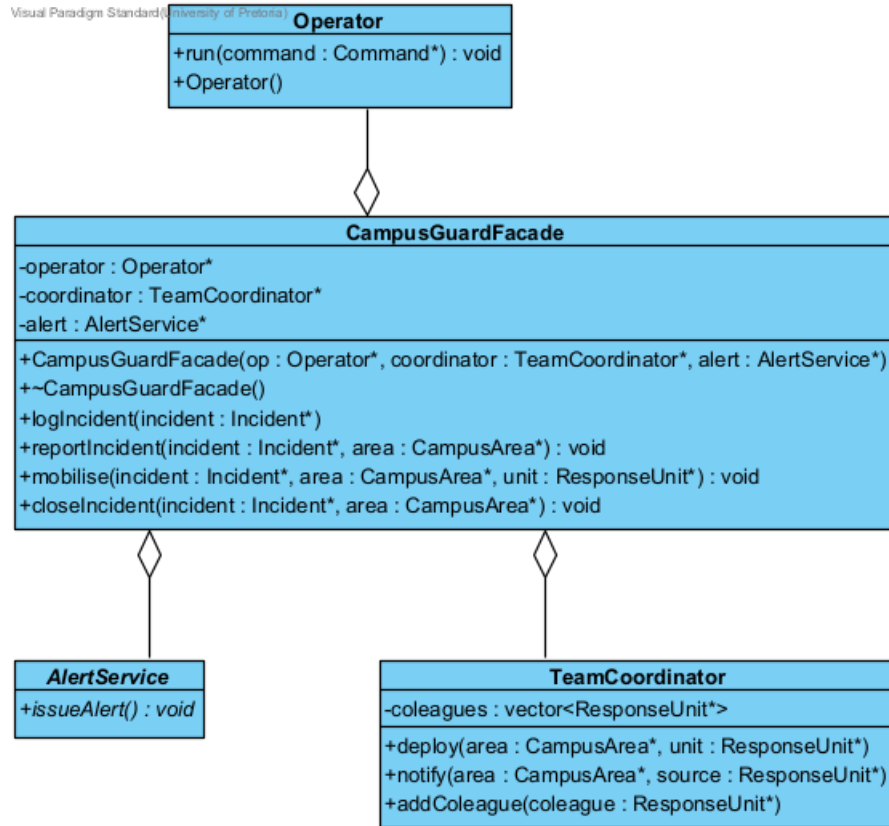


Figure 6: CampusGuardFacade with Operator, AlertService and TeamCoordinator

Sequence diagram: dispatch a unit (Command and Mediator)

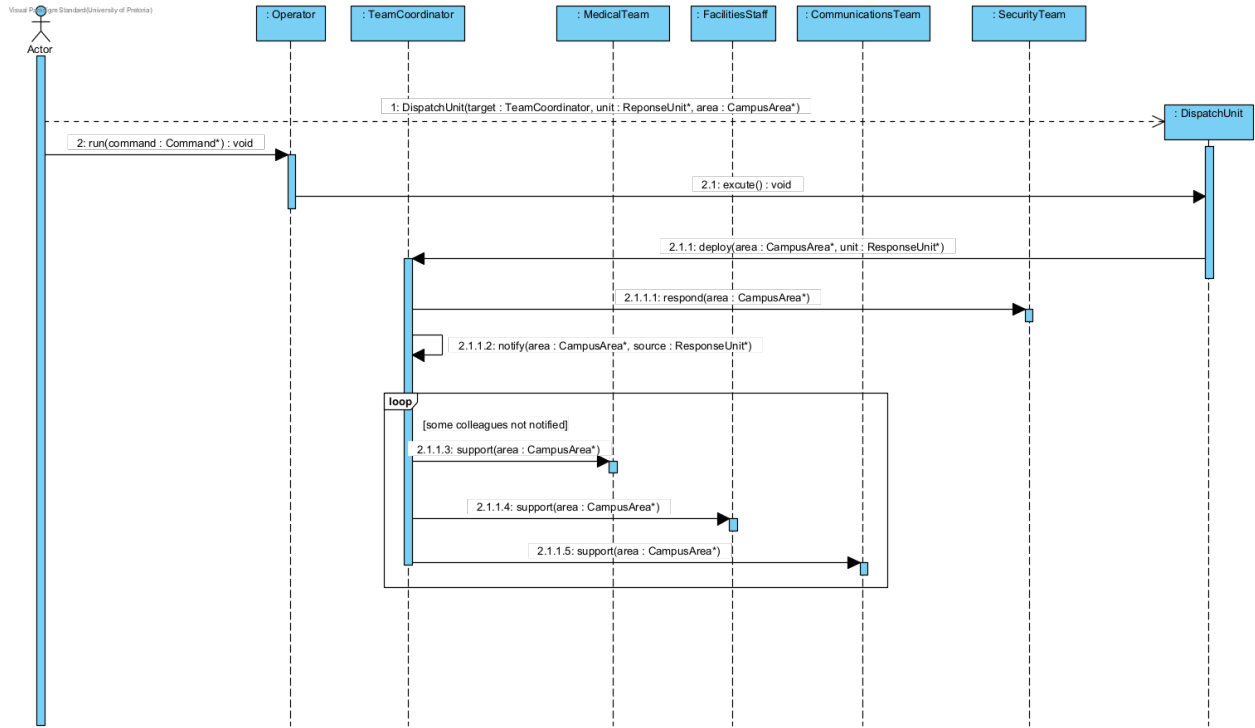


Figure 7: DispatchUnit through TeamCoordinator, then support from the other teams

Sequence diagram: report an incident (Facade)

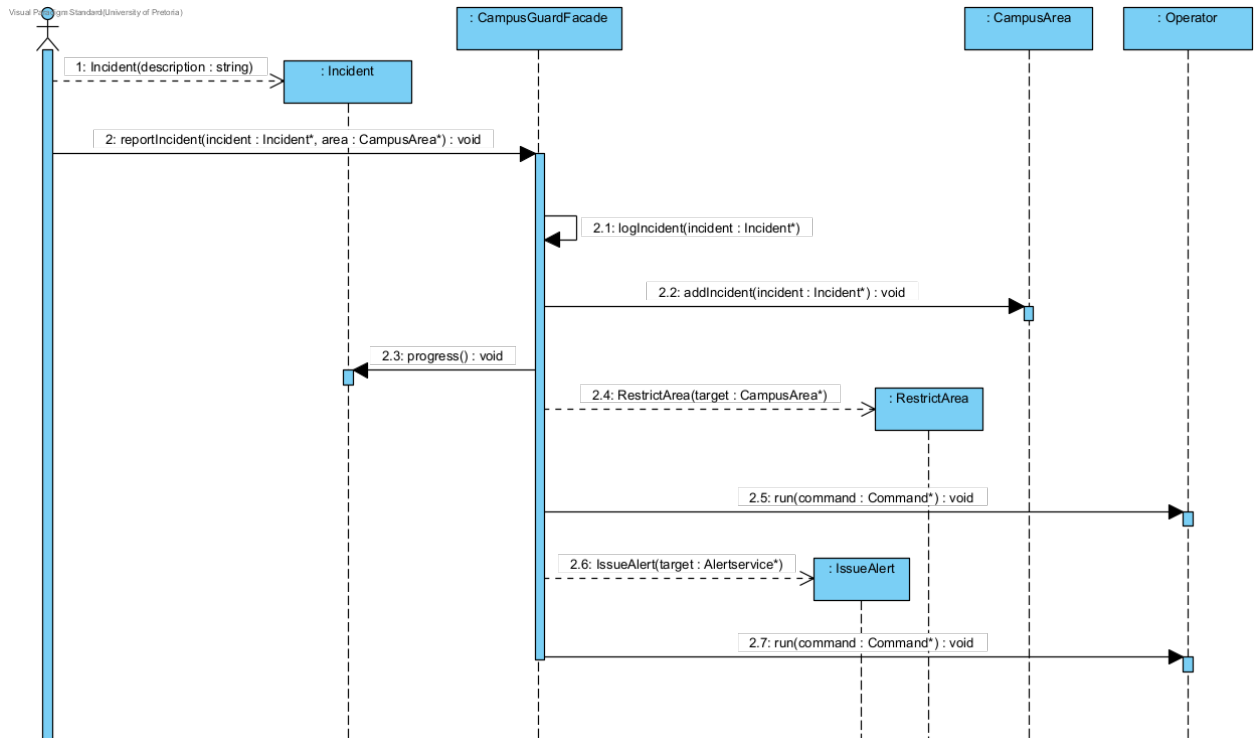


Figure 8: CampusGuardFacade reportIncident

State diagram: incident lifecycle

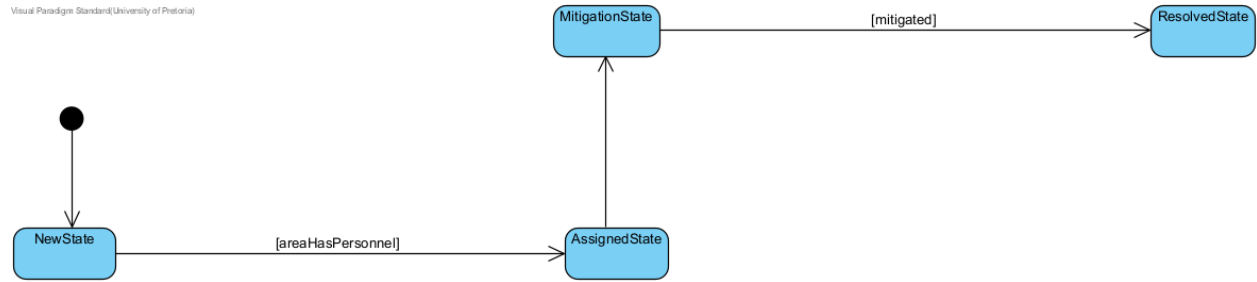


Figure 9: NewState, AssignedState, MitigationState and ResolvedState

Task 5: Engineering Quality

a. Docker demonstration

The assessed demonstration is launched from the repository root with:

```
docker compose up --build
```

- Stop the container with `docker compose down`.

b. Ownership policy

Every polymorphic base (`Command`, `Coordinator`, `ResponseUnit`, `AlertService`, `CampusArea`, `AccessState`, `IncidentState`) has a virtual destructor. Ownership is split so that only one object deletes each heap pointer.

`main` owns the campus map. Buildings are created with `new` and deleted at shutdown. Building owns its child `CampusArea` objects and deletes them in `~Building()`. Rooms do not own children.

Each `CampusArea` owns its current `AccessState`. The constructor allocates `OpenState`. `updateState` replaces the pointer and deletes the old state. The area destructor deletes the last state. Incidents and response units on an area are non-owning lists. The area does not `delete` them. On destruction it only clears the incident back-pointer so a surviving `Incident` does not keep a dangling area.

Each `Incident` owns its current `IncidentState` (`NewState` at construction). `updateState` deletes the previous state. The incident does not own its `CampusArea`.

`Operator` does not own commands. Scenario code constructs `DispatchUnit`, `RestrictArea`, `IssueAlert` and the lock/unlock commands on the stack (or as locals in the facade) and passes `Command*` into `run`. After `excute()` returns, the caller still owns that object.

`TeamCoordinator` does not own colleagues. Teams are stack objects in `main`. The coordinator only stores pointers and sets each unit's mediator pointer. `CampusGuardFacade` does not own the operator, coordinator or `AlertService`. `AlertServiceAdapter` does not own `LegacyAlertService`.

The rule: If you `new` it, the same owner `deletes` it (map, access state, incident state). If you only point at it (teams, commands, adapter, incidents on an area), you do not delete it.

c. GDB and Valgrind (Docker)

`gdb` and `valgrind` are installed in the image. After `docker compose up --build`, both are run inside that environment. `make` already uses `-g`.

GDB:

```
docker compose run --rm -it --entrypoint gdb campusguard ./campusguard
```

The Engineering fire could move from New to Assigned before anyone was dispatched.

```

(gdb) break NewState::updateState
Breakpoint 1 at NewState::updateState
(gdb) break Operator::run
Breakpoint 2 at Operator::run
(gdb) run
Breakpoint 1, NewState::updateState (this=...)
(gdb) print context->areaHasPersonnel()
$1 = false
(gdb) bt
#0 NewState::updateState
#1 Incident::progress
#2 main

```

`areaHasPersonnel()` was false on the first `fire.progress()` (no unit on Engineering yet) and the old code still moved to Assigned. The transition now stays in New until a unit is on the area.

```

Breakpoint 2, Operator::run (this=..., command=0x0)
(gdb) print command
$2 = (Command *) 0x0

```

A null `Command*` is rejected instead of calling `excute()` on `0x0`.

Valgrind:

```
docker compose run --rm --entrypoint make campusguard valgrind
```

That is `valgrind --leak-check=full --show-leak-kinds=all ./campusguard` inside the same image. The tutor can run that command during the demo. The process exits 0 after **CampusGuard terminated successfully**. Use the HEAP SUMMARY in that output as the leak evidence.

d. Git / GitHub history

Repository: https://github.com/patrickkaleo/COS214_P5

Work is on `main`, `dev` and `Task-2-Implement-the-Integrated-Model`. Pull request #1 merged the implementation branch. The history is from the start of the practical, not a single dump at the end.

Patrick Simuyemba ([patrickkaleo](#)) created the repo, folder layout, Task 1 write-up and first UML, later Makefile/Docker adjustments, incident-area coupling, and the state-transition guards above.

Shelby Bodenstein implemented most of the C++ model (commands, states, composite, mediator, adapter, facade, `main`), added the first Dockerfile and the `campusguard` Compose service, and opened the integration PR.

All three members have commits on the remote. The teaching team can follow design and implementation in that history.